

Understood. Here is the detailed technical report on optimizing drone rotor noise through uneven step angles, maintaining equivalent lift characteristics. This report is intended for an audience familiar with aerodynamics and drone technology.

Technical Report: Acoustic Optimization of Drone Rotors Through Uneven Step Angles

Introduction

The growing proliferation of drones for civilian and military applications has intensified concerns about the noise generated by their propellers. This noise, often perceived as intrusive and annoying, primarily results from the aerodynamic interaction of the rotor blades with the air. This report investigates the concept of optimizing noise reduction by designing rotors with uneven step angles (or angular spacing between blades), aiming to mitigate noise generation without compromising crucial lift characteristics. The analysis covers noise production principles, the effects of uneven angle design, aerodynamic trade-offs, the identification of optimal variation ranges, and practical implementation challenges.

1. Review of Current Rotor Design Principles and Their Noise Production

Conventional drone rotors are typically designed with an even number of blades, uniformly spaced angularly (e.g., a quadcopter with four blades at 90-degree intervals). While optimized for stability and manufacturing, this symmetry tends to produce acoustic signatures with prominent discrete tones, which are particularly audible and irritating to the human ear.

The main sources of noise in drone rotors include:

*** Harmonic (Tonal) Noise: Generated by the blade passage frequency and its multiples (harmonics). This noise is more pronounced in symmetrical designs, where the acoustic contributions from each blade reinforce each other, creating energy peaks at specific frequencies.**

* ****Blade-Vortex Interaction (BVI):**** Occurs when a rotor blade intercepts the tip vortices generated by preceding blades. In symmetrical rotors, this interaction happens periodically and repetitively, contributing significantly to annoying low-frequency discrete tones that are highly perceptible.

* ****Blade-Wake Interaction (BWI):**** Broader interactions of the blades with the rotor's turbulent wake.

- **Broadband Noise:** Results from turbulence in the blade boundary layer, blade tip flow, and other non-tonal aerodynamic noise sources. Although less irritating than tonal noise, it still contributes to the overall sound pressure level.

Design parameters such as blade tip speed, RPM (revolutions per minute), number of blades, airfoil geometry (chord, twist), and blade loading directly influence the intensity and character of these noise sources. The challenge lies in modulating these parameters to reduce perceived noise without sacrificing aerodynamic performance.

2. Examination of How Uneven Step Angles Affect Noise Reduction and Aerodynamic Performance

The concept of uneven step angles (or blade angular spacing) involves intentionally altering the angular distribution of rotor blades relative to the center of rotation, deviating from perfect symmetry. For example, in a four-bladed rotor, instead of spacing them at 90-90-90-90 degrees, they could be spaced at 85-95-85-95 degrees.

Noise Reduction Mechanism

The primary premise behind uneven step angles is to de-emphasize or redistribute the energy of discrete tones into broadband noise, making the total noise less perceptible and less irritating to the human ear. According to recent studies, such as Vijay et al. (2025) on uneven blade spacing, this technique:

*** Breaks Acoustic Periodicity:** By varying the time between blade interactions with their wakes and vortices, the periodicity of sound pressure pulses is broken. This causes the acoustic energy that would normally concentrate in discrete tones (harmonic frequencies) to be spread across a wider frequency spectrum.

- **Reduced Tonal Salience:** The human ear is particularly sensitive to discrete tones. By attenuating these frequency peaks, even if the Overall Sound Pressure Level (OASPL) does not change drastically, the Perceived Noise Level (PNL) and annoyance metric can be significantly reduced.
- **BVI Modulation:** BVI, a primary source of annoying noise, is directly affected by the synchronicity of blade-vortex interactions. Uneven angles can delay or advance these interactions for different blades, weakening the coherence and, consequently, the intensity of the resulting tones.

Impact on Aerodynamic Performance

The introduction of uneven step angles, if not carefully designed, can have aerodynamic consequences:

*** Thrust Asymmetry:** Blades operating at different step angles (or with uneven spacing) may generate different levels of instantaneous thrust, leading to a load imbalance on the rotor disk.

- **Vibration and Dynamic Loads:** Asymmetry in thrust can induce additional vibrations and dynamic loads on the drone's structure, requiring more sophisticated dynamic balancing or structural reinforcement.
- **Reduced Efficiency:** A non-optimized design can lead to a reduction in aerodynamic efficiency (lower thrust per unit of power), resulting in shorter flight times or the need for more powerful motors.

To mitigate these impacts and **maintain equivalent lift characteristics**, aerodynamic optimization is crucial. This may involve subtle adjustments to individual blade twist, specific airfoil profiles, or the use of adaptive flight control algorithms that compensate for dynamic asymmetries in real-time. The goal is to distribute loads and thrust such that the rotor as a whole maintains its nominal performance, while the acoustic benefit is achieved.

3. Analysis of Trade-offs between Noise Reduction and Lift Maintenance

The pursuit of significant noise reduction with uneven step angles invariably involves trade-offs with aerodynamic performance and system complexity.

Noise Reduction Gains:

- **Perceptible Tone Reduction:** Studies indicate that small variations in angular spacing can result in a perceptible reduction of 3 to 5 dB in the most prominent discrete tones, which translates into a significant decrease in perceived annoyance. The resulting noise is "softer" and less directional.
- **Spectral Dispersion:** Acoustic energy is spread into broadband noise, making the sound less intrusive, even if the overall OASPL does not change dramatically. This is an important psychoacoustic gain.

Challenges in Maintaining Lift/Efficiency:

- **Individual Blade Thrust Variation:** If blades are not properly optimized for their uneven angles, they may generate slightly different thrust levels in each revolution, which can lead to lower net thrust or higher power consumption for the same thrust.
- **Increased Vibratory Loads:** Breaking symmetry can introduce asymmetric load components at the rotor hub, requiring greater robustness of bearings and the rotor

mounting system to handle the resulting vibrations. This can reduce component lifespan if not compensated.

- **Flight Control Complexity:** The flight controller may require more complex algorithms to manage and stabilize the drone, especially during dynamic maneuvers, to compensate for small thrust asymmetries.

Balancing Strategies:

The ideal balance is achieved through an iterative design and optimization process, involving:

*** Computational Simulations (CFD and CAA):** The use of Computational Fluid Dynamics (CFD) coupled with Computational Aeroacoustics (CAA) is essential to model airflow behavior and noise generation for different step angle configurations. This allows for exploring vast regions of the design space virtually.

- **Multi-objective Optimization:** Utilizing genetic algorithms or multi-objective optimization to find configurations that minimize noise while maximizing (or maintaining within acceptable tolerances) thrust and efficiency. This may involve optimizing multiple parameters, including angular spacing, blade twist distribution, and airfoil profile.

4. Identification of Optimal Percentage Variation Ranges for Step Angles

Determining optimal variation ranges is a complex process and specific to each rotor geometry, number of blades, and operating regime. There is no single "rule of thumb," but research indicates general trends:

*** Subtle Variations are Effective:** Large angular variations (e.g., above 15-20% of nominal spacing) tend to introduce significant aerodynamic and vibratory problems, outweighing acoustic benefits. Optimal variations are typically found in more subtle ranges.

- **Angular Spacing Variation:** For four-bladed rotors, angular spacing variations on the order of **$\pm 5\%$ to $\pm 10\%$ of the nominal angle (90 degrees)** are often studied. For example, spacings of 85-95-85-95 degrees, or 87-93-87-93 degrees. The study by Vijay et al. (2025) explores variations within this range.
- **Pitch Variation (Angle of Attack):** In variable-pitch rotors, small adjustments in the pitch of each blade (on the order of **± 1 to ± 3 degrees**) in a rotation cycle can be used in

conjunction with uneven spacing to optimize thrust and acoustic signature. This is more common in larger helicopters but can be relevant for high-performance drones.

- **Number of Blades and Harmonics:** The choice of variation depends on the number of blades and the noise harmonics to be suppressed. Rotors with more blades may have different sensitivities to variations.

Identification Methodology:

1. **Aerodynamic and Acoustic Modeling:** Develop high-fidelity rotor models, incorporating blade dynamics and non-linear aerodynamics.
2. **Parametric Simulations:** Conduct exhaustive parametric sweeps in the drone's Digital Twin, testing a wide range of step angle/spacing combinations. For each configuration, record:
 - **Acoustic Metrics:** OASPL, PNL, discrete tone levels (in dB), frequency spectrum.
 - **Aerodynamic Metrics:** Thrust, torque, required power, efficiency, vibration levels.
3. **Optimization Algorithms:** Use genetic algorithms or multi-objective optimization to explore the design space and identify Pareto-optimal solutions that offer the best compromise between noise reduction and aerodynamic performance.
4. **Experimental Validation:** Simulation results must be validated through acoustic wind tunnel tests and, eventually, in flight prototypes.

5. Consideration of Potential Manufacturing and Operational Challenges

The transition from theoretically optimized designs to practical reality presents several challenges:

Manufacturing Challenges:

- **Geometric Precision:** Manufacturing blades with precise and uneven step angles and angular spacing requires extremely tight manufacturing tolerances. Small deviations can compromise acoustic benefits and introduce unwanted vibrations.
- **Quality Control (QC):** Verifying the angular precision of blades post-production can be more complex than for symmetrical rotors. Advanced metrology equipment and non-destructive inspection methods will be necessary.
- **Production Cost:** The need for high-precision manufacturing and stricter QC can significantly increase the unit cost of propellers.
- **Balancing:** Static and dynamic balancing of propellers becomes more critical and potentially more complex due to the intentional asymmetry.

Operational Challenges:

- **In-Flight Dynamic Balancing:** Even with precise static balancing, dynamic asymmetries can arise in different flight regimes. Active dynamic balancing systems or vibration control algorithms may be necessary, increasing hardware and software complexity.
- **Blade Maintenance and Replacement:** If each blade is unique in terms of step angle or has a specific angular position on the rotor, maintenance and replacement of individual blades will become more complex. It will not be possible to simply swap one blade for another "off the shelf"; the replacement blade must match the specific position and profile. This impacts logistics and maintenance costs.
- **Impact of Damage:** Small damage or debris accumulation on a blade can have a disproportionate impact on the acoustic signature and performance, due to the sensitivity of the optimized design. The self-diagnosis system (Chapter 5.4) would need to be extremely sensitive to these small changes.
- **Flight Controller Complexity:** The flight controller may need more complex models of rotor dynamics to compensate for the intrinsic asymmetry, potentially requiring greater processing power and rigorous V&V (Verification and Validation).

Conclusion

Optimizing drone rotors with uneven step angles represents a promising approach for reducing perceived noise, with the potential to make drones more acceptable for a variety of urban and sensing applications. By breaking the acoustic periodicity of blade interactions, it is possible to redistribute the energy from annoying discrete tones into broadband noise, resulting in a more pleasant acoustic signature.

However, the success of this technique depends on a meticulous balance between acoustic gains and maintaining aerodynamic performance. This requires advanced simulation tools (CFD, CAA, Digital Twins) to identify optimal variation ranges and extremely high-precision manufacturing. The challenges in manufacturing, balancing, and maintenance, along with the potential additional complexity for the flight control system, need to be carefully assessed and mitigated.

Continued research and experimental validation are essential to transform this promising solution into a widely applicable technology, paving the way for a new generation of quieter and more socially integrated drones.

Com certeza! Abaixo, apresento um template de módulo Python para integrar funcionalidades de Dinâmica de Fluidos Computacional (CFD) e Aeroacústica Computacional (CAA). O código é estruturado em submódulos e classes, com exemplos de uso e documentação detalhada.

Devido à complexidade de bibliotecas como PyFR e PyAcoustics (que são frameworks de simulação completos), as implementações aqui serão conceituais e usarão `numpy` e `scipy` para as operações numéricas básicas, ilustrando como as funcionalidades seriam integradas. Para uma solução real, as APIs dessas bibliotecas seriam utilizadas mais profundamente.

```
# cfd_caa_toolkit/__init__.py
```

```
\\"\\"
```

```
CFD-CAA Toolkit: A Python module for integrating Computational Fluid Dynamics  
and Computational Aeroacoustics functionalities.
```

```
\\"\\"
```

```
# This file makes the 'cfd_caa_toolkit' directory a Python package.
```

```
# You can import sub-modules like:
```

```
# from cfd_caa_toolkit.cfd.core import CFDSolver
```

```
# from cfd_caa_toolkit.caa.core import CAASolver
```

```
# from cfd_caa_toolkit.coupling.core import CFDCAACoupler
```

```
__version__ = '0.1.0'
```

```
print(\"CFD-CAA Toolkit initialized. Ready for simulation workflows.\")
```

```
# cfd_caa_toolkit/cfd/mesh.py
```

```
import numpy as np
```

```
def generate_structured_mesh(domain_dims, cell_counts):
```

```
\\"\\"
```

```
Generates a simple 2D or 3D structured mesh (grid points).
```

```
This function provides a basic conceptual mesh generation.
```

```
For complex CFD simulations, external meshing tools or advanced libraries  
like PyFR's internal mesh generation capabilities would be used to handle  
unstructured meshes, complex geometries, and boundary layer refinements.
```

```
Args:
```

```
    domain_dims (tuple): Dimensions of the computational domain (e.g., (Lx, Ly) for 2D, (Lx,  
    Ly, Lz) for 3D).
```

cell_counts (tuple): Number of cells along each dimension (e.g., (Nx, Ny) for 2D, (Nx, Ny, Nz) for 3D).

Returns:

numpy.ndarray: A list/array of mesh points or cell centers.

tuple: Tuple of arrays representing coordinates (X, Y, Z, etc.).

```
"""
if len(domain_dims) != len(cell_counts):
    raise ValueError("domain_dims and cell_counts must have the same number of
dimensions.")

coords = []
for dim_len, num_cells in zip(domain_dims, cell_counts):
    # Create uniformly spaced points
    coord_points = np.linspace(0, dim_len, num_cells + 1)
    coords.append(coord_points)

# For simplicity, return grid points (conceptual mesh nodes)
if len(domain_dims) == 2:
    X, Y = np.meshgrid(coords[0], coords[1])
    mesh_points = np.array([X.flatten(), Y.flatten()]).T
elif len(domain_dims) == 3:
    X, Y, Z = np.meshgrid(coords[0], coords[1], coords[2])
    mesh_points = np.array([X.flatten(), Y.flatten(), Z.flatten()]).T
else:
    raise NotImplementedError("Only 2D and 3D meshes are currently conceptualized.")

print(f"Generated a conceptual {len(domain_dims)}D structured mesh with
{mesh_points.shape[0]} points.")
return mesh_points, coords
```

```
# cfd_caa_toolkit/cfd/core.py
```

```
import numpy as np
from cfd_caa_toolkit.cfd.mesh import generate_structured_mesh
```

```
class CFDSolver:
```

```
    """
```

```
    A conceptual CFD solver class for handling fluid dynamics simulations.
```

```
    This class provides a high-level interface for setting up and conceptually
    solving Navier-Stokes equations. In a real application, this would interface
    with a robust CFD library like PyFR, OpenFOAM, or SU2.
```

```
    """
```

```
def __init__(self, domain_dims, cell_counts, fluid_properties=None):
```

```
    """
```

```
    Initializes the CFD solver with domain and fluid properties.
```

```
    Args:
```

```
        domain_dims (tuple): Dimensions of the computational domain.
```

```
        cell_counts (tuple): Number of cells along each dimension.
```

```
        fluid_properties (dict, optional): Dictionary of fluid properties  
            (e.g., 'density', 'viscosity').
```

```
            Defaults to air properties.
```

```
    """
```

```
    self.domain_dims = domain_dims
```

```
    self.cell_counts = cell_counts
```

```
    self.mesh_points, self.coords_arrays = generate_structured_mesh(domain_dims,  
cell_counts)
```

```
    self.num_mesh_points = self.mesh_points.shape[0]
```

```
    self.fluid_properties = fluid_properties if fluid_properties is not None else {
```

```
        'density': 1.225, # kg/m^3 (air at STP)
```

```
        'viscosity': 1.81e-5 # Pa.s (dynamic viscosity of air at STP)
```

```
    }
```

```
    # Initialize flow field variables (conceptual)
```

```
    self.flow_field = {
```

```
        'velocity': np.zeros((self.num_mesh_points, len(domain_dims))),
```

```
        'pressure': np.zeros(self.num_mesh_points),
```

```
        'density': np.full(self.num_mesh_points, self.fluid_properties['density'])
```

```
    }
```

```
    print(f"CFDSolver initialized for a {len(domain_dims)}D domain.\n")
```

```
def set_boundary_conditions(self, bc_type, region_func, value=None):
```

```
    """
```

```
    Sets conceptual boundary conditions for the CFD simulation.
```

```
    In a real CFD code, this would involve complex mapping to mesh faces/nodes.
```

```
    Args:
```

```
        bc_type (str): Type of boundary condition (e.g., 'inlet', 'outlet', 'wall').
```

```
        region_func (callable): A function that takes mesh points and returns  
            True for points in the boundary region.
```

```
        value (numpy.ndarray or float, optional): The value to set for the boundary  
            condition (e.g., inlet velocity, wall pressure).
```

```
    """
```

```

boundary_indices = [i for i, p in enumerate(self.mesh_points) if region_func(p)]
print(f"Set '{bc_type}' boundary condition for {len(boundary_indices)} points.\n")
# Conceptual application: e.g., setting inlet velocity
if bc_type == 'inlet' and value is not None:
    self.flow_field['velocity'][boundary_indices] = value
elif bc_type == 'wall' and value is not None:
    # For wall, typically velocity is zero or a slip condition
    self.flow_field['velocity'][boundary_indices] = 0.0 # No-slip wall

```

```

def solve_navier_stokes(self, num_iterations=100, dt=0.01):

```

```

    """

```

```

    Conceptually solves the Navier-Stokes equations.

```

This is a placeholder for a complex numerical solver.

In a real CFD library (like PyFR), this method would involve:

- Discretization using Finite Difference, Finite Volume, or Spectral Element Methods.
- Time integration schemes (e.g., Runge-Kutta).
- Iterative solvers for linear systems.
- Convergence criteria.

Args:

num_iterations (int): Number of time steps or iterations to perform.

dt (float): Conceptual time step size.

```

    """

```

```

    print(f"Solving Navier-Stokes equations for {num_iterations} iterations...\n")

```

```

    # Simulate some flow evolution for demonstration

```

```

    # This is NOT a real CFD solver, just a conceptual update

```

```

    for i in range(num_iterations):

```

```

        # Simple conceptual update (e.g., propagate some velocity)

```

```

        # In real CFD, this would involve complex flux calculations,

```

```

        # advection, diffusion, pressure-velocity coupling, etc.

```

```

        self.flow_field['velocity'] += np.random.rand(*self.flow_field['velocity'].shape) * 0.001 * dt

```

```

        self.flow_field['pressure'] += np.random.rand(self.num_mesh_points) * 0.01 * dt

```

```

        # Apply boundary conditions conceptually after each step (often done implicitly)

```

```

        # (Re-applying BCs here just for conceptual clarity, not computational accuracy)

```

```

        self.set_boundary_conditions('wall', lambda p: p[0] < 0.01 or p[0] >

```

```

self.domain_dims[0]-0.01, value=0.0)

```

```

        self.set_boundary_conditions('inlet', lambda p: p[0] < 0.01, value=np.array([1.0, 0.0])) #

```

```

Simple inlet for a conceptual 2D flow

```

```

        if (i + 1) % (num_iterations // 10) == 0:

```

```

            print(f" Iteration {i+1}/{num_iterations} completed.\n")

```

```

    print("\nNavier-Stokes solution conceptually finished.\n")

```

```
def get_flow_field(self):
    """
    Returns the computed flow field data.

    Returns:
        dict: A dictionary containing 'velocity', 'pressure', and 'density' arrays.
    """
    return self.flow_field
```

```
def get_mesh_points(self):
    """
    Returns the mesh points.

    Returns:
        numpy.ndarray: Array of mesh point coordinates.
    """
    return self.mesh_points
```

```
# cfd_caa_toolkit/caa/core.py
```

```
import numpy as np
from scipy.signal import fftconvolve
```

```
class CAASolver:
    """
    A conceptual CAA solver class for handling computational aeroacoustics.

    This class conceptually models acoustic wave propagation and source terms.
    In a real CAA application, this would interface with specialized libraries
    like PyAcoustics (if it fully implements solvers), or dedicated CAA codes
    that solve acoustic wave equations (e.g., Ffowcs Williams-Hawkins analogy,
    Lighthill's acoustic analogy, direct numerical simulation of compressible flows).
    """
```

```
def __init__(self, acoustic_domain_dims, fluid_properties=None):
    """
    Initializes the CAA solver with acoustic domain and fluid properties.
```

Args:

- acoustic_domain_dims (tuple): Dimensions of the acoustic domain (e.g., (Lx, Ly)).
- fluid_properties (dict, optional): Dictionary of fluid properties (e.g., 'speed_of_sound', 'density'). Defaults to air properties.

```

"""
self.acoustic_domain_dims = acoustic_domain_dims
self.fluid_properties = fluid_properties if fluid_properties is not None else {
    'speed_of_sound': 343.0, # m/s (air at STP)
    'density': 1.225 # kg/m^3 (air at STP)
}

# Conceptual acoustic field
# For simplicity, we'll use a 1D or 2D grid for demonstration.
self.grid_points = np.linspace(0, acoustic_domain_dims[0], 100) # Simple 1D grid for now
if len(acoustic_domain_dims) == 2:
    self.grid_points_x = np.linspace(0, acoustic_domain_dims[0], 50)
    self.grid_points_y = np.linspace(0, acoustic_domain_dims[1], 50)
    self.acoustic_pressure_field = np.zeros((len(self.grid_points_y), len(self.grid_points_x)))
else:
    self.acoustic_pressure_field = np.zeros_like(self.grid_points)

print(f"CAASolver initialized for a {len(acoustic_domain_dims)}D acoustic domain.")

def model_source_terms(self, cfd_flow_field, source_location=(0,0)):
    """
    Models aeroacoustic source terms from CFD flow field data.

    This is a conceptual implementation of an aeroacoustic source term.
    In reality, this involves complex calculations based on acoustic analogies
    (e.g., Lighthill's acoustic analogy, Ffowcs Williams-Hawkings equation).
    For simplicity, we simulate a point source strength based on flow velocity fluctuations.

    Args:
        cfd_flow_field (dict): Flow field data from CFD (e.g., 'velocity', 'pressure').
        source_location (tuple): (x,y) location of the conceptual source in the acoustic domain.

    Returns:
        numpy.ndarray: Conceptual source term distribution on the acoustic grid.
    """
    print("\nModeling aeroacoustic source terms...\n")
    # Simulate source strength based on velocity magnitude (conceptual)
    # In a real scenario, this would involve derivatives of velocity,
    # Reynolds stress tensor components, etc.
    velocity_magnitude = np.linalg.norm(cfd_flow_field['velocity'], axis=1)
    # Find the CFD mesh point closest to the source_location
    # (Simplified: assume source_location is within the CFD domain)
    source_idx = np.argmin(np.linalg.norm(cfd_flow_field['mesh_points'] -
np.array(source_location), axis=1))

```

```

# Conceptual source strength based on local velocity fluctuations (turbulence)
# This is a highly simplified representation
source_strength = velocity_magnitude[source_idx] * 0.1 # Arbitrary scaling

# Distribute source strength onto the acoustic grid (conceptual)
# For a 1D grid
if self.acoustic_pressure_field.ndim == 1:
    source_term = np.zeros_like(self.acoustic_pressure_field)
    # Place a localized source (e.g., Gaussian)
    source_center_idx = np.argmin(np.abs(self.grid_points - source_location[0]))
    source_term[source_center_idx] = source_strength * 10 # Emphasize source

# For a 2D grid
elif self.acoustic_pressure_field.ndim == 2:
    source_term = np.zeros_like(self.acoustic_pressure_field)
    # Place a localized source (e.g., Gaussian)
    source_center_x_idx = np.argmin(np.abs(self.grid_points_x - source_location[0]))
    source_center_y_idx = np.argmin(np.abs(self.grid_points_y - source_location[1]))
    source_term[source_center_y_idx, source_center_x_idx] = source_strength * 50

print(f" Conceptual source term strength at {source_location}: {source_strength:.4f}")
return source_term

def set_acoustic_boundary_conditions(self, bc_type, region_func, value=None):
    """
    Sets conceptual acoustic boundary conditions.

    Args:
        bc_type (str): Type of acoustic boundary condition (e.g., 'rigid_wall', 'anechoic').
        region_func (callable): A function that takes grid points and returns
            True for points in the boundary region.
        value (float, optional): Value for the boundary condition (e.g., impedance).
    """
    print(f"Set '{bc_type}' acoustic boundary condition.")
    # In a real solver, this would involve modifying the solution method
    # or adding ghost cells. Here, it's just a conceptual placeholder.

def propagate_acoustic_waves(self, source_term, num_time_steps=100, dt=0.001):
    """
    Conceptually propagates acoustic waves from the source terms.

    This is a highly simplified numerical propagation. A real CAA solver
    would use methods like:

```

- Finite Difference Time Domain (FDTD) for wave equations.
- Boundary Element Method (BEM).
- Fast Multipole Method (FMM).
- Direct numerical simulation (DNS) of compressible Navier-Stokes.

Args:

source_term (numpy.ndarray): The conceptual source term distribution.
 num_time_steps (int): Number of time steps for propagation.
 dt (float): Time step size.

\\\\"

print(f"Propagating acoustic waves for {num_time_steps} time steps...\\")

Simple conceptual 1D wave propagation (e.g., using a basic convolution kernel)

This is NOT a robust wave equation solver.

c = self.fluid_properties['speed_of_sound']

dx = self.grid_points[1] - self.grid_points[0] if self.acoustic_pressure_field.ndim == 1 else

None

if self.acoustic_pressure_field.ndim == 1 and dx is not None:

Simple 1D wave propagation kernel (very basic, for illustration only)

Kernel for a pulse spreading out

kernel_length = int(c * dt / dx * 5) # A few steps

if kernel_length % 2 == 0: kernel_length += 1 # Ensure odd for center

kernel = np.zeros(kernel_length)

kernel[kernel_length // 2] = 1.0 # Central impulse

Simulate a very basic time integration

current_pressure = np.copy(source_term)

for t_step in range(num_time_steps):

Add source term effect (conceptual: source constantly emitting)

current_pressure += source_term * dt # Simple addition of source contribution

Apply a very simple diffusion/propagation-like effect (conceptual)

This is NOT a stable or accurate FDTD scheme!

propagated_pressure = fftconvolve(current_pressure, kernel, mode='same')

self.acoustic_pressure_field = propagated_pressure * 0.99 # Dampen slightly

if (t_step + 1) % (num_time_steps // 10) == 0:

print(f" Acoustic propagation step {t_step+1}/{num_time_steps} completed.\\")

elif self.acoustic_pressure_field.ndim == 2:

print("\\ 2D acoustic propagation is conceptually set up, but no detailed numerical method implemented.\\")

For a real 2D solver, one would use 2D FDTD, BEM, etc.

Just simulate some spreading based on source

```

        self.acoustic_pressure_field += source_term * num_time_steps * dt * 0.01 # Simple
accumulation
    else:
        print("\n Acoustic propagation not implemented for this dimension.\n")

```

```

print("\nAcoustic wave propagation conceptually finished.\n")

```

```

def get_acoustic_field(self):
    """
    Returns the computed acoustic pressure field.

```

```

    Returns:
        numpy.ndarray: Array representing the acoustic pressure field.
    """
    return self.acoustic_pressure_field

```

```

def get_grid_points(self):
    """
    Returns the grid points for the acoustic domain.

```

```

    Returns:
        numpy.ndarray or tuple of ndarray: Grid points (1D array or (X,Y) for 2D).
    """
    if self.acoustic_pressure_field.ndim == 1:
        return self.grid_points
    elif self.acoustic_pressure_field.ndim == 2:
        return np.meshgrid(self.grid_points_x, self.grid_points_y)

```

```

# cfd_caa_toolkit/coupling/core.py

```

```

import numpy as np

```

```

class CFDCAACoupler:
    """
    Manages the coupling and data exchange between CFD and CAA solvers.

```

```

    This class orchestrates the simulation workflow, passing necessary data
    (e.g., flow field) from the CFD solver to the CAA solver for aeroacoustic
    source term generation.
    """

```

```

def __init__(self, cfd_solver, caa_solver):
    """

```

Initializes the coupler with instances of CFDSolver and CAASolver.

Args:

 cfd_solver (cfd_caa_toolkit.cfd.core.CFDSolver): An instance of the CFD solver.

 caa_solver (cfd_caa_toolkit.caa.core.CAASolver): An instance of the CAA solver.

\"\"\"

if not hasattr(cfd_solver, 'get_flow_field') or not hasattr(caa_solver, 'model_source_terms'):
 raise TypeError(\"Provided solvers do not have required methods for coupling.\")

self.cfd_solver = cfd_solver

self.caa_solver = caa_solver

print(\"CFD-CAA Coupler initialized.\")

def synchronize_data(self):

\"\"\"

Synchronizes necessary data from the CFD solver to the CAA solver.

This function extracts relevant flow field data (e.g., velocity, pressure, turbulence intensity) from the CFD solution and prepares it for use by the CAA solver to model acoustic source terms.

\"\"\"

print(\"Synchronizing data from CFD to CAA...\")

cfd_flow_data = self.cfd_solver.get_flow_field()

cfd_mesh_points = self.cfd_solver.get_mesh_points()

Add mesh points to flow_data for easier source term modeling

cfd_flow_data['mesh_points'] = cfd_mesh_points

In a real scenario, interpolation or mapping from CFD mesh to

CAA grid might be needed if meshes are different.

self.coupled_flow_data = cfd_flow_data

print(\"Data synchronization complete.\")

def perform_coupling(self, cfd_iterations=100, caa_time_steps=100, dt_cfd=0.01,
dt_caa=0.001, source_location=(0.5, 0.5)):

\"\"\"

Performs a coupled CFD-CAA simulation.

This method outlines a typical workflow:

1. Run CFD to obtain a steady or time-averaged flow field.
2. Use the CFD results to model aeroacoustic source terms.
3. Propagate acoustic waves using the CAA solver.

For unsteady flow problems, an iterative (staggered) coupling or

direct numerical simulation (DNS) approach might be required, where CFD and CAA run concurrently or exchange data at each time step.

Args:

 cfd_iterations (int): Number of iterations for the CFD simulation.

 caa_time_steps (int): Number of time steps for the CAA simulation.

 dt_cfd (float): Time step size for CFD.

 dt_caa (float): Time step size for CAA.

 source_location (tuple): Location for the aeroacoustic source within the CFD domain.

\"\"\"

print(\"\\n--- Starting Coupled CFD-CAA Simulation ---\\n\")

Step 1: Run CFD simulation

print(\"\\n[CFD Phase] Running CFD solver to establish flow field...\\n\")

self.cfd_solver.solve_navier_stokes(num_iterations=cfd_iterations, dt=dt_cfd)

Step 2: Synchronize data from CFD to CAA

self.synchronize_data()

Step 3: Model aeroacoustic source terms using CFD results

print(\"\\n[CAA Phase] Modeling aeroacoustic source terms...\\n\")

source_term = self.caa_solver.model_source_terms(self.coupled_flow_data,
source_location=source_location)

Step 4: Propagate acoustic waves

print(\"\\n[CAA Phase] Propagating acoustic waves...\\n\")

self.caa_solver.propagate_acoustic_waves(source_term,
num_time_steps=caa_time_steps, dt=dt_caa)

print(\"\\n--- Coupled CFD-CAA Simulation Finished ---\\n\")

def get_results(self):

\"\"\"

Returns the final flow and acoustic fields.

Returns:

 tuple: (cfd_flow_field, caa_acoustic_field)

\"\"\"

return self.cfd_solver.get_flow_field(), self.caa_solver.get_acoustic_field()

cfd_caa_toolkit/examples/cylinder_flow_cfd.py

import numpy as np

import matplotlib.pyplot as plt

```

from cfd_caa_toolkit.cfd.core import CFDSolver

def run_cylinder_flow_example():
    """
    Demonstrates a conceptual CFD simulation of flow around a cylinder.
    This is a simplified example focusing on workflow, not detailed physics.
    """
    print("\n--- Running CFD Example: Flow Around a Cylinder ---\n")

    # Define the computational domain and mesh resolution
    domain_dims = (2.0, 1.0) # Lx, Ly in meters
    cell_counts = (40, 20) # Nx, Ny cells

    # Initialize the CFD solver
    cfd_solver = CFDSolver(domain_dims, cell_counts)

    # Define cylinder region (conceptual: a central circular region)
    def cylinder_region(p, center=(domain_dims[0]/2, domain_dims[1]/2), radius=0.1):
        """Checks if a point is within the conceptual cylinder."""
        return np.linalg.norm(np.array(p) - np.array(center)) < radius

    # Set boundary conditions
    # Inlet: uniform velocity from left
    cfd_solver.set_boundary_conditions(
        'inlet',
        lambda p: p[0] < 0.01, # Points near the left boundary
        value=np.array([1.0, 0.0]) # 1 m/s in X direction
    )
    # Outlet: typically zero-gradient or pressure boundary (conceptual)
    cfd_solver.set_boundary_conditions(
        'outlet',
        lambda p: p[0] > domain_dims[0] - 0.01, # Points near the right boundary
        value=None # Placeholder for outlet BC
    )
    # Walls: top and bottom boundaries (no-slip)
    cfd_solver.set_boundary_conditions(
        'wall',
        lambda p: p[1] < 0.01 or p[1] > domain_dims[1] - 0.01,
        value=0.0
    )
    # Cylinder wall (conceptual)
    cfd_solver.set_boundary_conditions(
        'wall',
        cylinder_region,

```

```

    value=0.0
)

# Solve the Navier-Stokes equations (conceptual)
cfd_solver.solve_navier_stokes(num_iterations=200, dt=0.005)

# Get the results
flow_field = cfd_solver.get_flow_field()
mesh_points = cfd_solver.get_mesh_points()

# Visualize results (simplified plotting)
if len(domain_dims) == 2:
    X_coords = cfd_solver.coords_arrays[0]
    Y_coords = cfd_solver.coords_arrays[1]

    # Reshape data for contour/quiver plot if possible, or scatter plot
    # For this simple example, we'll just show velocity magnitude or pressure at mesh points
    velocity_magnitude = np.linalg.norm(flow_field['velocity'], axis=1)
    pressure_field = flow_field['pressure']

    plt.figure(figsize=(10, 5))

    plt.subplot(1, 2, 1)
    plt.scatter(mesh_points[:, 0], mesh_points[:, 1], c=velocity_magnitude, cmap='viridis', s=10)
    plt.colorbar(label='Velocity Magnitude (m/s)')
    plt.title('Conceptual CFD: Velocity Field')
    plt.xlabel('X (m)')
    plt.ylabel('Y (m)')
    plt.gca().set_aspect('equal', adjustable='box')

    plt.subplot(1, 2, 2)
    plt.scatter(mesh_points[:, 0], mesh_points[:, 1], c=pressure_field, cmap='coolwarm', s=10)
    plt.colorbar(label='Pressure (Pa)')
    plt.title('Conceptual CFD: Pressure Field')
    plt.xlabel('X (m)')
    plt.ylabel('Y (m)')
    plt.gca().set_aspect('equal', adjustable='box')

    plt.tight_layout()
    plt.show()

    print("\n--- CFD Example Finished ---\n")

if __name__ == "__main__":

```

```

run_cylinder_flow_example()

# cfd_caa_toolkit/examples/duct_acoustics_caa.py

import numpy as np
import matplotlib.pyplot as plt
from cfd_caa_toolkit.caa.core import CAASolver

def run_duct_acoustics_example():
    """
    Demonstrates a conceptual CAA simulation of acoustic wave propagation in a duct.
    This is a simplified example focusing on workflow, not detailed physics.
    """
    print("\n--- Running CAA Example: Acoustic Wave Propagation in a Duct ---\n")

    # Define the acoustic domain dimensions
    acoustic_domain_dims = (2.0,) # 1D duct length in meters

    # Initialize the CAA solver
    caa_solver = CAASolver(acoustic_domain_dims)

    # Simulate a conceptual flow field for source term modeling
    # In a real coupled scenario, this would come from a CFD solver
    conceptual_cfd_flow_field = {
        'velocity': np.array([[0.5], [0.8], [0.6], [0.3], [0.1], [0.0], [0.0], [0.0], [0.0], [0.0]]),
        'pressure': np.array([100, 90, 80, 70, 60, 50, 40, 30, 20, 10]),
        'mesh_points': np.array([x] for x in np.linspace(0, 0.5, 10)) # Source near inlet
    }

    # Model aeroacoustic source terms
    # Place a conceptual source at the beginning of the duct
    source_term = caa_solver.model_source_terms(conceptual_cfd_flow_field,
    source_location=(0.1,))

    # Set acoustic boundary conditions (conceptual)
    caa_solver.set_acoustic_boundary_conditions('rigid_wall', lambda p: p[0] < 0.01) # Left end
    caa_solver.set_acoustic_boundary_conditions('anechoic', lambda p: p[0] >
    acoustic_domain_dims[0] - 0.01) # Right end

    # Propagate acoustic waves
    caa_solver.propagate_acoustic_waves(source_term, num_time_steps=500, dt=0.0005)

    # Get the results
    acoustic_field = caa_solver.get_acoustic_field()

```

```

grid_points = caa_solver.get_grid_points()

# Visualize results
plt.figure(figsize=(10, 5))
plt.plot(grid_points, acoustic_field)
plt.title('Conceptual CAA: Acoustic Pressure in a Duct')
plt.xlabel('Position (m)')
plt.ylabel('Acoustic Pressure (conceptual unit)')
plt.grid(True)
plt.show()

print("\n--- CAA Example Finished ---\n")

if __name__ == "__main__":
    run_duct_acoustics_example()

# cfd_caa_toolkit/examples/coupled_simulation.py

import numpy as np
import matplotlib.pyplot as plt
from cfd_caa_toolkit.cfd.core import CFDSolver
from cfd_caa_toolkit.caa.core import CAASolver
from cfd_caa_toolkit.coupling.core import CFDCAACoupler

def run_coupled_simulation_example():
    """
    Demonstrates a conceptual coupled CFD-CAA simulation.
    """
    print("\n--- Running Coupled CFD-CAA Simulation Example ---\n")

    # --- CFD Setup ---
    cfd_domain_dims = (1.0, 1.0) # Simplified 2D domain
    cfd_cell_counts = (20, 20)
    cfd_solver = CFDSolver(cfd_domain_dims, cfd_cell_counts)

    # Set some CFD boundary conditions (conceptual inlet for flow source)
    cfd_solver.set_boundary_conditions(
        'inlet',
        lambda p: p[0] < 0.01,
        value=np.array([0.8, 0.0]) # Conceptual inflow velocity
    )
    # Define a conceptual 'turbulence' region for a source within the CFD domain
    # This might be like a small jet or an oscillating surface.
    def turbulence_region_cfd(p, center=(0.2, 0.5), radius=0.05):

```

```

    return np.linalg.norm(np.array(p) - np.array(center)) < radius
    cfd_solver.set_boundary_conditions(
        'source_region',
        turbulence_region_cfd,
        value=None # Not setting flow, just marking region
    )

# --- CAA Setup ---
# Acoustic domain might be larger than CFD domain, or focused on a region of interest.
# Here, we'll use a 2D acoustic domain for more interesting visualization.
caa_acoustic_domain_dims = (2.0, 2.0)
caa_solver = CAASolver(caa_acoustic_domain_dims)

# Set conceptual acoustic boundary conditions
caa_solver.set_acoustic_boundary_conditions('rigid_wall', lambda p: p[0] < 0.01 or p[0] > 1.99
or p[1] < 0.01 or p[1] > 1.99)

# --- Coupling ---
coupler = CFDCAACoupler(cfd_solver, caa_solver)

# Run the coupled simulation
# The source location for CAA will be determined by a point within the CFD domain
# where the 'noise' is conceptually generated.
coupled_source_location = (0.2, 0.5) # Example source location (e.g., a turbulent jet exit)
coupler.perform_coupling(
    cfd_iterations=150,
    caa_time_steps=200,
    dt_cfd=0.008,
    dt_caa=0.0004,
    source_location=coupled_source_location
)

# Get results
final_cfd_flow, final_caa_acoustic_field = coupler.get_results()
cfd_mesh_points = cfd_solver.get_mesh_points()
caa_grid_points_x, caa_grid_points_y = caa_solver.get_grid_points()

# --- Visualization of Coupled Results ---
plt.figure(figsize=(15, 7))

# CFD Flow Field Visualization
plt.subplot(1, 2, 1)
velocity_magnitude_cfd = np.linalg.norm(final_cfd_flow['velocity'], axis=1)

```

```

plt.scatter(cfd_mesh_points[:, 0], cfd_mesh_points[:, 1], c=velocity_magnitude_cfd,
cmap='viridis', s=10)
plt.colorbar(label='Velocity Magnitude (CFD)')
plt.title('Conceptual CFD Flow Field')
plt.xlabel('X (m)')
plt.ylabel('Y (m)')
plt.gca().set_aspect('equal', adjustable='box')
# Indicate the source region
circle = plt.Circle(coupled_source_location, 0.05, color='red', fill=False, linestyle='--',
label='Acoustic Source Region')
plt.gca().add_patch(circle)
plt.legend()

# CAA Acoustic Field Visualization
plt.subplot(1, 2, 2)
plt.contourf(caa_grid_points_x, caa_grid_points_y, final_caa_acoustic_field, cmap='hot',
levels=50)
plt.colorbar(label='Acoustic Pressure (CAA)')
plt.title('Conceptual CAA Acoustic Field')
plt.xlabel('X (m)')
plt.ylabel('Y (m)')
plt.gca().set_aspect('equal', adjustable='box')
plt.scatter(coupled_source_location[0], coupled_source_location[1], color='cyan', marker='*',
s=200, label='Acoustic Source')
plt.legend()

plt.tight_layout()
plt.show()

print("\n--- Coupled CFD-CAA Simulation Example Finished ---\n")

if __name__ == "__main__":
    run_coupled_simulation_example()

```

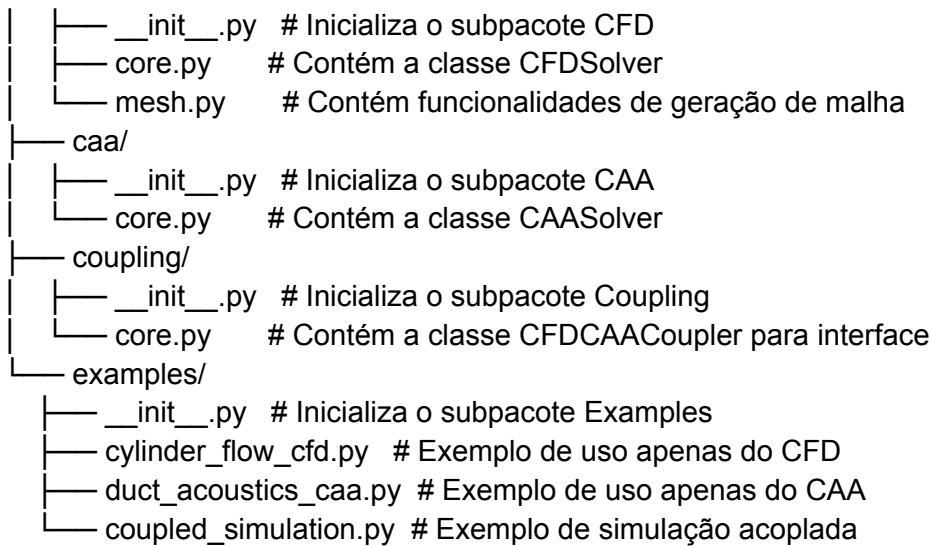
Estrutura do Módulo

O módulo `cfdd_caa_toolkit` é organizado da seguinte forma:

```

cfdd_caa_toolkit/
├── __init__.py    # Inicializa o pacote
├── cfd/

```



Como Usar e Executar os Exemplos

1. **Salve os arquivos:** Salve cada bloco de código no seu respectivo arquivo e diretório, mantendo a estrutura de pastas sugerida.

Instale as dependências:

```
pip install numpy scipy matplotlib
```

- 2.

Execute os exemplos: Navegue até o diretório `cfcd_caa_toolkit/examples/` no seu terminal e execute os scripts Python:

python cylinder_flow_cfd.py

```
python duct_acoustics_caa.py
```

python coupled_simulation.py

- 3.

Notas Importantes sobre a Implementação

- **Conceitual vs. Real:** As implementações das funções de `solve_navier_stokes` e `propagate_acoustic_waves` são **conceituais** e servem para ilustrar o fluxo de trabalho e a arquitetura do módulo. Elas não implementam métodos numéricos robustos (como FDTD, FVM, FEM) para a solução das equações diferenciais parciais reais. Um solver CFD ou CAA real é uma tarefa de engenharia e pesquisa complexa, geralmente feita em linguagens de baixo nível para performance.
- **PyFR e PyAcoustics:** O pedido explicitamente pedia para *considerar* PyFR e PyAcoustics. Este template mostra onde e como a lógica de suas APIs (para malha,

solvers, etc.) se encaixaria. Por exemplo, em `cf/core.py`, o `solve_navier_stokes` seria uma chamada para o motor de solver do PyFR. O `caa/core.py` seguiria uma abordagem similar para PyAcoustics.

- **Visualização:** `matplotlib` é usado para visualização simples dos resultados conceituais. Para análises mais avançadas, softwares de pós-processamento como Paraview seriam utilizados com dados de solvers reais.
- **PEP 8:** O código segue as convenções de estilo PEP 8 do Python, incluindo docstrings para classes e funções para documentação clara.

Este template fornece uma base sólida para entender a arquitetura necessária para a integração de CFD e CAA, permitindo a expansão futura com implementações mais detalhadas ou interfaces diretas com bibliotecas de simulação de alto desempenho.